

Stage 3: Design Thinking & Architecture Report

CodeScribed Assessment

1. Executive Summary

In this assessment we are asked to build a multi-modal AI system bridging **Computer Vision** and **Natural Language Processing**.

Users upload a face image → the system detects emotion → synthesizes narrative context → enables Q&A via **RAG (Retrieval-Augmented Generation)**.

The system integrates:

- Deep Learning (CNNs) for perception
 - Generative AI (LLMs) for reasoning
-

2. System Architecture

A modular, linear pipeline with high explainability.

High-Level Data Flow

```
[Input Image]
↓
[Face Detection & Crop]
↓
[Emotion Classification]
↓
[Context Synthesis]
↓
[Vector Embedding]
↓
[RAG Retrieval]
```

↓

[LLM Response]

Component Breakdown

A. Vision Layer (Perception)

- **Model:** ResNet18 (Fine-Tuned)
- **Dataset:** FER-2013
- **Preprocessing:** OpenCV Haar Cascades

Logic

- Detect face using Haar Cascades (`minNeighbors=8`)
 - Crop + normalize to **224×224**
 - If no face → fallback to full-frame analysis
 - ResNet18 predicts **one of 7 emotions**:
 - Angry
 - Disgust
 - Fear
 - Happy
 - Sad
 - Surprise
 - Neutral
-

B. Bridge Layer (Synthesis)

- **Role:** Convert emotion label → narrative context
 - **Mechanism:** Rule-based template generation
 - **Output:** Synthetic *customer reviews* matching emotion
 - Example for *Sad*: "I am disappointed and heartbroken."
-

C. RAG System (Knowledge & Reasoning)

- **Orchestrator:** LangChain
- **Embeddings:** `sentence-transformers/all-MiniLM-L6-v2`
- **Vector Store:** FAISS (in-memory; low latency)
- **LLM:** Google **Flan-T5-Large**

- Chosen for better reasoning vs. T5-Small (yet CPU-friendly)

Generation Controls

- `repetition_penalty = 1.5`
- `temperature = 0.6`
- `top_p = 0.95`

These prevent repetitive or hallucinated outputs.

D. Application Layer (UI)

- **Framework:** Streamlit
 - **Functions:** Upload image, visualize inference, chat
 - **Optimization:**
 - Uses `@st.cache_resource` → loads models only once
-

3. Technology Stack Rationale

Component	Choice	Justification
Deep Learning	PyTorch	Dynamic graph → easy debugging
Backbone	ResNet18	Residual connections prevent vanishing gradients
Orchestration	LangChain	Industry-standard for RAG
Vector DB	FAISS	Fast, local, no cloud dependency
LLM	Flan-T5	Open-source, CPU-friendly, instruction-tuned

4. Scalability & Production Roadmap

These things can be done it's just ideas that can be done to scale this application .

1. Decoupling into Microservices

- **Vision Service:**
 - Docker container running ResNet inference (gRPC endpoint).

- **RAG Service:**

Independent text generation module.

2. Vector Database Migration

- **Current:** FAISS (in-memory)
 - **Production:** Pinecone / Milvus
 - Persistence
 - Sharding
 - Metadata search (e.g., angry reviews from “yesterday”)
-

3. Caching Strategy

- Integrate **Redis** to store:
 - Reused embeddings
 - Template-based reviews
-

4. Hybrid Search

Combine:

- Dense vector search (FAISS)
- Keyword search (BM25)

Improves retrieval for specific terms (e.g., “refund”, “shipping”).

5. Ethics, Bias & Robustness

Ethical Implications

- **Consent:**

Emotion detection must be opt-in; avoid surveillance misuse.
 - **Emotional Complexity:**

Single static image ≠ real emotional state.
-

Bias Mitigation

- **Vision Bias:**

FER-2013 is demographically imbalanced → retrain on **FairFace**.

- **Hallucination Control:**

RAG prompt + repetition penalty prevent fabricated info.

System Robustness

- **Fallback Logic:**

If Haar Cascade fails → process full image.

- Ensures reliable output even under occlusion or poor lighting.